

EA4. Documentación de la Arquitectura y Modelo de Datos

Pipeline de Ingesta, Preprocesamiento y Enriquecimiento
en un Entorno Simulado de Big Data en la Nube

Estudiante: Justin Beckham Cardona Yepes

Asignatura: Infraestructura y Arquitectura para Big Data

Institución: Institución Universitaria Digital de Antioquia (IU Digital)

Repositorio: github.com/justinbeckham1204/justin_cardona

0 Introducción

Este documento consolida y explica la arquitectura técnica del proyecto integrador desarrollado a lo largo de las actividades EA1, EA2 y EA3, así como el modelo de datos resultante del proceso de integración. El proyecto simula un entorno de **Big Data en la nube** mediante un pipeline autónomo compuesto por tres fases secuenciales —ingesta, preprocesamiento/limpieza y enriquecimiento heterogéneo—, orquestadas mediante un flujo de integración continua (CI/CD) mediante GitHub Actions, y soportadas por una base de datos analítica en SQLite.

El objetivo de este documento es evidenciar, de forma clara y trazable, la estructura general de la solución, los flujos de datos entre componentes, la selección de herramientas, el mecanismo de automatización, y el modelo de datos final (esquema, relaciones y estructura), justificando las decisiones de diseño tomadas y su aporte a la escalabilidad y mantenibilidad del sistema.

1 Descripción General de la Arquitectura

1.1 Visión Global

La arquitectura del proyecto integrador simula un entorno de procesamiento de datos en la nube utilizando componentes ligeros pero representativos de un pipeline real de Big Data: una fuente externa de datos (API pública), un motor de procesamiento en Python (Pandas), un almacén analítico persistente (SQLite) y un orquestador de automatización (GitHub Actions) que actúa como capa de "cómputo administrado" (equivalente conceptual a un scheduler o a un servicio serverless en un proveedor cloud real).

El pipeline se organiza en tres fases secuenciales y desacopladas, cada una correspondiente a una entrega del proyecto integrador (EA1, EA2 y EA3), pero diseñadas para ejecutarse de manera encadenada y autónoma:

1. **Fase de Ingesta (EA1):** extracción de datos de todos los países del mundo desde la API pública `REST Countries`, transformación a registros planos y persistencia inicial en la base de datos relacional analítica SQLite (`src/db/ingestion.db`), además de una muestra de auditoría en Excel.
2. **Fase de Preprocesamiento y Limpieza (EA2):** carga de los datos crudos desde SQLite, análisis exploratorio, eliminación de duplicados por el identificador único de país (`cca3`), imputación de valores nulos, corrección de tipos de datos y normalización Min-Max de variables numéricas, generando un conjunto de datos limpio y un reporte de auditoría.
3. **Fase de Enriquecimiento (EA3):** integración relacional (Left Join por `cca3`) del conjunto de datos limpio con seis fuentes externas heterogéneas en distintos formatos (JSON, CSV, XML, HTML, TXT y XLSX), produciendo un dataset unificado y enriquecido, junto con su reporte de trazabilidad.

Todo el pipeline se ejecuta de manera automática mediante un workflow de GitHub Actions, que simula el comportamiento de un entorno cloud administrado: aprovisiona una máquina virtual efímera, instala las dependencias necesarias, ejecuta los scripts en orden y persiste los artefactos generados (bases de datos, archivos Excel y reportes de auditoría) de vuelta en el repositorio mediante un commit automático.

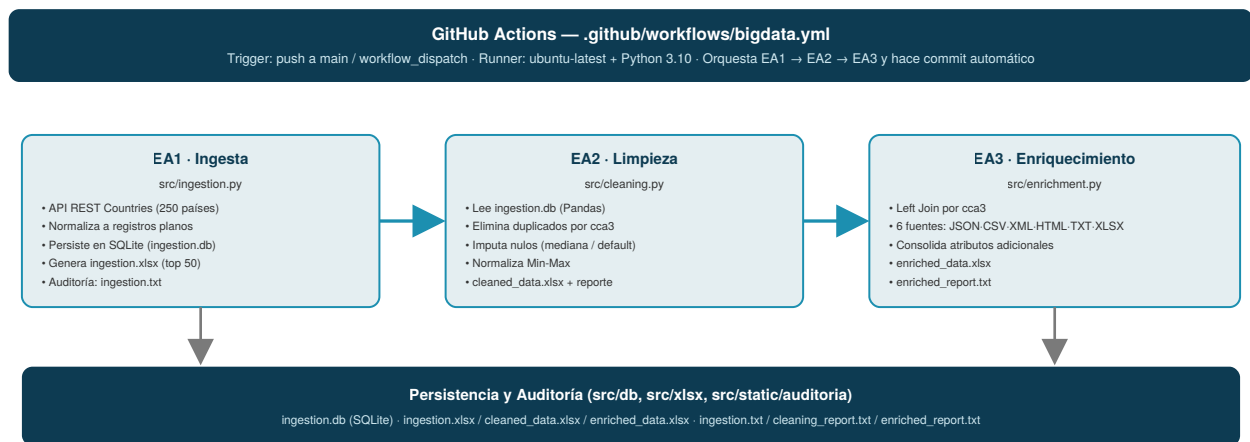


Figura 1. Flujo general de la arquitectura: de la extracción del API a la persistencia final, orquestado por GitHub Actions.

1.2 Componentes Principales

Componente	Rol en la arquitectura	Ubicación en el repositorio
API REST Countries	Fuente de datos externa (capa de ingesta). Provee información estructurada en JSON sobre los 250 países del mundo.	Consumida vía HTTP en <code>src/ingestion.py</code>
Script de ingesta	Extrae, aplanar y carga los datos crudos en la base de datos analítica; genera evidencias de auditoría.	<code>src/ingestion.py</code>
Base de datos analítica (SQLite)	Almacén persistente central del pipeline; simula el data warehouse / bucket analítico del entorno cloud.	<code>src/db/ingestion.db</code>
Script de limpieza	Preprocesa los datos crudos: deduplicación, imputación, tipado y normalización.	<code>src/cleaning.py</code>
Script de enriquecimiento	Integra 6 fuentes multiformato mediante Left Join por <code>cca3</code> , generando el dataset analítico final.	<code>src/enrichment.py</code>
Mecanismo de automatización	Orquesta la ejecución secuencial del pipeline y persiste los artefactos resultantes sin intervención manual.	<code>.github/workflows/bigdata.yml</code>
Artefactos de auditoría	Evidencian trazabilidad: conteos, duplicados, nulos tratados y conciliación entre fuentes.	<code>src/static/auditoria/*.txt</code>

2 Diagramas de Arquitectura

2.1 Flujo de Extracción y Almacenamiento (EA1)

El siguiente diagrama detalla el subproceso de ingesta: la petición HTTP al API, el aplanamiento del JSON anidado a un esquema tabular, y la doble persistencia (base de datos SQLite y archivo Excel de auditoría).

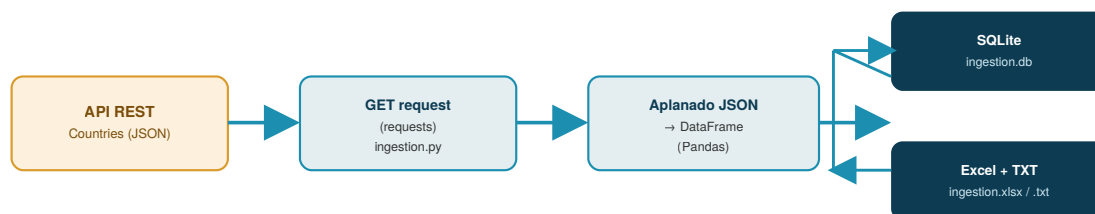


Figura 2. Flujo detallado de ingesta: de la respuesta del API a la doble persistencia (SQLite + Excel/auditoría).

2.2 Flujo de Preprocesamiento y Enriquecimiento (EA2 → EA3)

El siguiente diagrama ilustra cómo los datos limpios de EA2 se convierten en la tabla base sobre la cual se aplican seis operaciones de *Left Join* (una por cada fuente externa) para construir el dataset enriquecido de EA3.

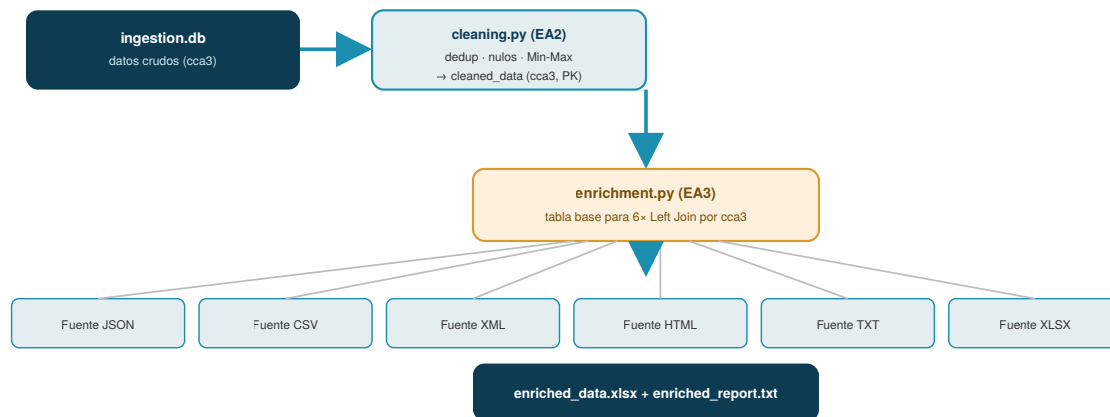


Figura 3. De los datos limpios (EA2) a la integración multiformato por *Left Join* sobre `cca3` (EA3).

Ambos diagramas fueron elaborados siguiendo la notación de flujo de datos utilizada en herramientas como draw.io (cajas de proceso, cilindros/paralelogramos de almacenamiento y flechas direccionales etiquetadas), y están etiquetados con el nombre exacto de cada script y artefacto tal como aparece en el repositorio, para garantizar trazabilidad directa entre el documento y el código.

3 Modelo de Datos

3.1 Definición del Esquema

El modelo de datos resultante del proceso de integración se organiza como un **esquema relacional en capas**, donde cada etapa del pipeline (ingesta, limpieza, enriquecimiento) produce una tabla propia, todas ellas conectadas por la llave `cca3` (código ISO 3166-1 alfa-3 del país), que actúa como identificador único y estable a lo largo de todo el proceso. Este enfoque permite trazar cualquier registro final hasta su origen crudo, cumpliendo el requisito de auditoría del proyecto.

Tabla: `países_ingesta` (EA1 — datos crudos)

Campo	Tipo	Descripción
<code>cca3</code> PK	TEXT (3)	Código ISO alfa-3 del país (identificador único).
<code>name_common</code>	TEXT	Nombre común del país.
<code>name_oficial</code>	TEXT	Nombre oficial del país.
<code>region</code>	TEXT	Región geográfica (p. ej. Americas, Europe).
<code>subregion</code>	TEXT	Subregión geográfica.
<code>capital</code>	TEXT	Capital del país.
<code>population</code>	INTEGER	Población total reportada por el API.
<code>area</code>	REAL	Área en km².
<code>languages</code>	TEXT	Idiomas oficiales (lista serializada).
<code>currencies</code>	TEXT	Monedas oficiales (lista serializada).
<code>independent</code>	BOOLEAN	Indicador de estado independiente.
<code>fecha_ingesta</code>	TEXT (ISO)	Marca de tiempo de la extracción.

Tabla: `países_limpios` (EA2 — datos preprocesados)

Campo	Tipo	Descripción
<code>cca3</code> PK FK	TEXT (3)	Referencia a <code>países_ingesta.cca3</code> ; deduplicado.
<code>name_common</code>	TEXT	Nombre común, sin nulos (valor por defecto si aplica).
<code>region</code>	TEXT	Región, con imputación de valor por defecto.

population	INTEGER	Población con nulos imputados por mediana.
area	REAL	Área con nulos imputados por mediana.
population_norm	REAL [0-1]	Población normalizada (Min-Max).
area_norm	REAL [0-1]	Área normalizada (Min-Max).
fecha_procesamiento	TEXT (ISO)	Marca de tiempo de la limpieza.

Tablas de fuentes externas (EA3 — un registro por país, cada una con `cca3` como PK/FK)

Tabla	Formato origen	Atributos que aporta al modelo
fuelle_json	JSON	Indicadores estructurados adicionales por país (p. ej. datos macroeconómicos o demográficos publicados como JSON).
fuelle_csv	CSV	Métricas tabulares complementarias por país.
fuelle_xml	XML	Atributos jerárquicos adicionales, aplanados durante el parseo.
fuelle_html	HTML	Datos extraídos de tablas publicadas en páginas web (web scraping / parsing de tablas HTML).
fuelle_txt	TXT	Datos semiestructurados de texto plano, parseados a pares clave-valor por país.
fuelle_xlsx	XLSX	Datos tabulares adicionales provenientes de hojas de cálculo.

Nota de trazabilidad: los nombres exactos de columnas de las seis fuentes externas dependen de los datasets específicos utilizados en `src/enrichment.py`. Se recomienda ajustar los nombres de campo de esta subsección con los nombres reales definidos en el script antes de la entrega final, para que el documento sea 100% consistente con el código fuente.

Tabla: `países_enriquecidos` (EA3 — resultado final)

Campo	Tipo	Descripción
<code>cca3</code> PK FK	TEXT (3)	Referencia a <code>países_limpios.cca3</code> .
(todos los campos de <code>países_limpios</code>)	—	Heredados vía Left Join.
(atributos de <code>fuelle_json</code> ... <code>fuelle_xlsx</code>)	—	Incorporados vía 6 operaciones Left Join sucesivas sobre <code>cca3</code> .
	TEXT (ISO)	Marca de tiempo del proceso de integración.

3.2 Relaciones entre Tablas

Todas las relaciones del modelo son de tipo **uno a uno (1:1)** sobre la llave `cca3`, dado que cada tabla representa exactamente un registro por país en cada etapa del pipeline. La tabla `países_ingesta` actúa como tabla raíz; `países_limpios` referencia a la anterior mediante clave foránea `cca3`; las seis tablas de fuentes externas son independientes entre sí pero comparten la misma llave; y `países_enriquecidos` es el resultado de aplicar seis Left Join consecutivos entre `países_limpios` y cada una de las fuentes, conservando siempre el universo completo de países definido en la etapa de limpieza (de ahí el uso de Left Join en lugar de Inner Join).

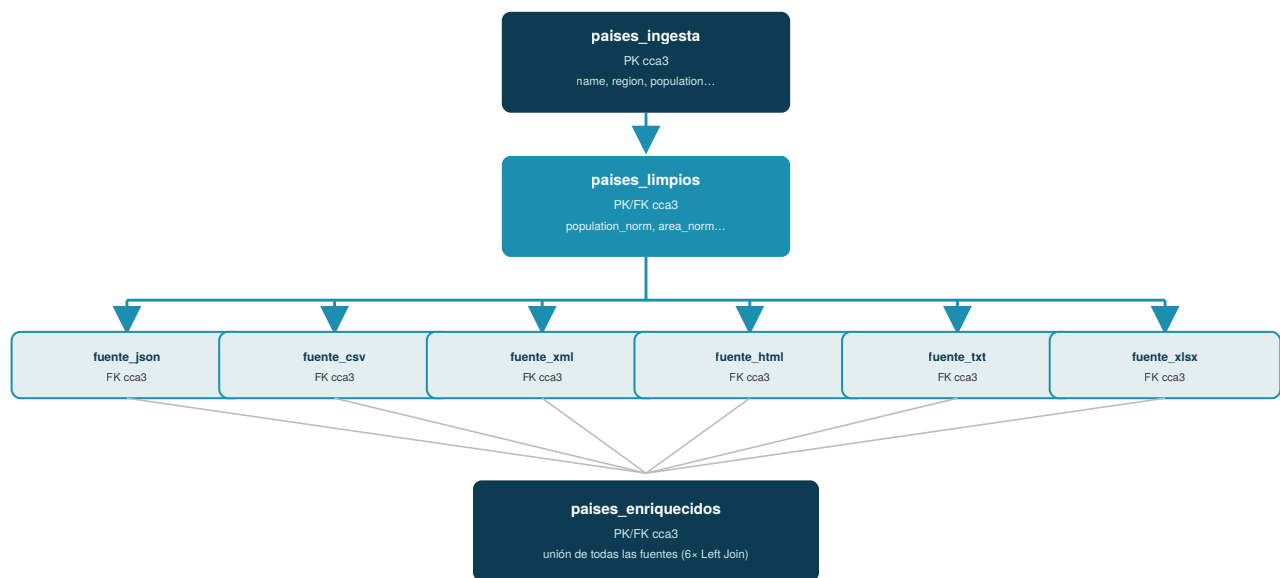


Figura 4. Diagrama Entidad-Relación (ER) del modelo de datos: relaciones 1:1 sobre `cca3` entre las etapas del pipeline.

3.3 Justificación del Diseño

El modelo se diseñó en capas (ingesta → limpieza → fuentes externas → enriquecido) en lugar de sobrescribir una única tabla, por tres razones principales:

- **Trazabilidad:** permite auditar en qué etapa se originó o modificó cada valor, lo cual es coherente con los reportes de auditoría (`ingestion.txt`, `cleaning_report.txt`, `enriched_report.txt`) que el proyecto genera en cada fase.
- **Desacoplamiento:** cada script (`ingestion.py`, `cleaning.py`, `enrichment.py`) puede ejecutarse y probarse de forma independiente, ya que consume la tabla de la etapa anterior sin necesidad de recalculiar todo el pipeline.

- **Consistencia de integración:** el uso de `cca3` como llave primaria/foránea universal —un estándar ISO estable y no ambiguo— evita los problemas de calidad de datos típicos de integrar por nombre de país (variantes de escritura, idioma, tildes), facilitando que las seis fuentes heterogéneas se integren de forma determinística mediante Left Join.

El uso de *Left Join* (y no *Inner Join*) en la fase de enriquecimiento es intencional: garantiza que ningún país del conjunto limpio se pierda aunque alguna fuente externa no tenga información para él, dejando esos campos como nulos en lugar de excluir el registro. Esto facilita el análisis posterior, ya que el analista dispone siempre del universo completo de países con la mayor cantidad de información disponible por registro.

4 Justificación de Herramientas y Tecnologías

4.1 Elección de Herramientas

Herramienta	Rol y justificación
SQLite	Se eligió como base de datos analítica central por ser un motor relacional embebido, sin necesidad de un servidor dedicado, que soporta SQL estándar y transacciones ACID. Para un proyecto integrador cuyo volumen de datos es acotado (cientos de países), SQLite ofrece la persistencia y la capacidad de consulta estructurada de una base de datos real, manteniendo el pipeline portable (un único archivo <code>.db</code> versionable) y fácil de ejecutar dentro de un runner efímero de GitHub Actions, sin costos ni configuración de infraestructura.
Pandas	Es el motor de procesamiento de todas las fases (ingesta, limpieza, enriquecimiento). Su API de DataFrames permite aplanar JSON anidado, leer/escribir múltiples formatos (SQL, Excel, CSV, XML, HTML), realizar deduplicación, imputación de nulos, normalización Min-Max y operaciones de <i>join</i> relacional (<code>merge</code>) de forma expresiva y con pocas líneas de código, lo cual acelera el desarrollo y facilita el mantenimiento del proyecto.
PySpark	Aunque la implementación actual del proyecto opera con Pandas dado el volumen de datos manejable, PySpark se justifica como la herramienta natural de evolución si el volumen de países, fuentes o histórico creciera varios órdenes de magnitud (por ejemplo, datos a nivel de ciudad/municipio en lugar de país, o registros históricos diarios). PySpark permitiría paralelizar las mismas operaciones de limpieza y Left Join sobre un clúster distribuido sin cambiar la lógica de negocio, ya que su API de DataFrames es conceptualmente análoga a la de Pandas. Se documenta aquí como parte de la estrategia de escalabilidad del proyecto (ver sección 6).
GitHub Actions	Se seleccionó como mecanismo de automatización por integrarse nativamente con el repositorio (sin necesidad de un orquestador externo), permitir definir triggers por evento (<code>push</code>) o manuales (<code>workflow_dispatch</code>), y aprovisionar en segundos una máquina virtual efímera con el entorno de Python necesario. Esto simula de forma fidedigna el comportamiento de un servicio de automatización cloud (equivalente conceptual a un job de Airflow, un Lambda con EventBridge o un pipeline de Azure DevOps), manteniendo el proyecto 100% reproducible y gratuito para fines académicos.

4.2 Simulación del Entorno Cloud

El entorno cloud se simula combinando tres elementos: **(1)** la base de datos SQLite generada y versionada a partir de las Actividades 1, 2 y 3, que cumple el rol de almacén analítico persistente accesible por todos los componentes del pipeline; **(2)** los scripts de procesamiento (`ingestion.py` , `cleaning.py` , `enrichment.py`), que representan las funciones de cómputo (equivalentes a funciones serverless o jobs batch) que transforman los datos en cada etapa; y **(3)** el runner efímero de GitHub Actions (`ubuntu-latest`), que se aprovisiona bajo demanda, ejecuta el pipeline completo y se

destruye al finalizar, exactamente como ocurriría con una instancia de cómputo temporal en un proveedor cloud real (AWS, Azure o GCP). El resultado del procesamiento se persiste de vuelta en el repositorio mediante un commit automático, cumpliendo el rol que en un entorno productivo tendría un bucket de almacenamiento o un data lake.

5 Flujo de Datos y Automatización

5.1 Explicación del Flujo

El flujo de datos completo, desde la ingesta hasta la construcción del modelo de datos final, sigue la siguiente secuencia:

1. **Disparo del workflow:** un `push` a la rama `main` o una ejecución manual (`workflow_dispatch`) inicia el pipeline.
2. **Aprovisionamiento:** GitHub Actions levanta una máquina virtual `ubuntu-latest` con Python 3.10 e instala las dependencias (`pandas` , `openpyxl` , `requests` , `lxml` , `html5lib`).
3. **Ingesta (EA1):** `ingestion.py` consulta el API REST Countries y persiste los datos crudos en `ingestion.db` , generando evidencias de auditoría.
4. **Limpieza (EA2):** `cleaning.py` lee de `ingestion.db` , aplica deduplicación, imputación y normalización, y exporta `cleaned_data.xlsx` junto con su reporte.
5. **Enriquecimiento (EA3):** `enrichment.py` integra las seis fuentes multiformato mediante Left Join sobre `cca3` , produciendo `enriched_data.xlsx` y `enriched_report.txt` .
6. **Persistencia automática:** si se detectan cambios en los artefactos generados, el workflow ejecuta `git add` , `git commit` y `git push` de forma autónoma, dejando el repositorio actualizado sin intervención manual.

Este flujo garantiza que el pipeline completo sea **reproducible** (cualquier persona puede clonar el repositorio y regenerar los mismos resultados) y **autónomo** (no depende de que un desarrollador ejecute los scripts manualmente cada vez que cambian los datos fuente).

6.1 Beneficios de la Arquitectura Propuesta

- El diseño por capas (ingesta → limpieza → enriquecimiento) facilita la auditoría y el mantenimiento independiente de cada etapa.
- El uso de `cca3` como llave universal simplifica la integración de fuentes heterogéneas y evita ambigüedades de nombres.
- La automatización con GitHub Actions elimina la intervención manual y garantiza reproducibilidad total del pipeline.
- SQLite y Pandas ofrecen una combinación liviana, gratuita y suficiente para el volumen de datos del proyecto académico.

6.2 Limitaciones

- SQLite no soporta concurrencia de escritura ni escalabilidad horizontal, por lo que no sería adecuado para volúmenes de datos de producción a gran escala.
- El procesamiento con Pandas ocurre en memoria de una sola máquina, lo que limita el tamaño de los datasets que el pipeline puede manejar eficientemente.
- El runner gratuito de GitHub Actions tiene límites de tiempo de ejecución y de minutos mensuales, lo cual restringe su uso como orquestador en un entorno productivo real.
- La ausencia de un esquema de versionado de datos (por ejemplo, particionado por fecha) limita la trazabilidad histórica más allá de la última ejecución.

6.3 Recomendaciones para un Entorno Real de Nube

- Migrar el almacén analítico de SQLite a un motor administrado (por ejemplo, Amazon RDS, Google BigQuery o Azure SQL) para soportar concurrencia y mayor volumen.
- Sustituir Pandas por PySpark cuando el volumen de datos o fuentes crezca significativamente, aprovechando el procesamiento distribuido sin reescribir la lógica de negocio.
- Reemplazar GitHub Actions por un orquestador especializado (Apache Airflow, AWS Step Functions o Azure Data Factory) para pipelines con dependencias más complejas o ejecución programada de alta frecuencia.
- Incorporar particionado por fecha de carga y versionado de esquema para mantener trazabilidad histórica completa del dataset enriquecido.
- Añadir pruebas automatizadas de calidad de datos (por ejemplo, con Great Expectations) dentro del propio workflow de CI/CD.